

Vyuha: A Reproducible, Free-Compute Layered Guardrail for Deploying LLM Defenses — and How to Measure One Honestly

Anonymous EACL submission

October 7, 2026

Abstract

Prompt injection is the top risk in the OWASP Top 10 for LLM Applications, and recent adaptive-attack work shows any single filter, once known, can be evaded — “the attacker moves second.” Yet most guard evaluations run on balanced laboratory sets, not deployment conditions. We present **Vyuha**, a defense-in-depth guardrail that composes six independent layers (normalisation, a CPU fast detector, a content guard, agent/tool-injection defense, output moderation, and continuous red-teaming) and ships as a library, a service, and a CLI. Our contribution is not a new algorithm but a *deployable composition*, an *evaluation methodology* suited to LLM-guard components (recall-first, confidence intervals, adaptive-attack ablation, a standardised-benchmark reconstruction, and base-rate discipline), and *full reproducibility on a single free GPU*. Measured head-on as a whole system at matched false-positive rate against Granite Guardian 4.1-8B, Vyuha catches more jailbreaks (+0.20) and disguised requests (+0.37) on in-distribution pools at about a third of its compute. Out of distribution, only the second gain survives: on unseen obfuscations Vyuha beats both Granite 4.1 (+0.65) and Llama Guard 3-8B (+0.61), but on adversarial jailbreaks, JailbreakBench artifacts and plain harmful requests it ties Granite and trails Llama Guard 3 (by 0.25 on JailbreakBench), and a Best-of-N augmentation attacker evades it as easily as either small guard. At the layer level, with 95% confidence intervals: the CPU fast layer matches a GPU neural guard’s obfuscation robustness at ~ 8 ms; against an adaptive attacker, normalisation alone leaves ASR at 1.00 while adversarial augmentation closes it to 0.03 at 1% false-

positive rate; on a NIST-AI-RMF guard benchmark the 0.6B content guard Vyuha binds reaches 0.886 recall on complete-harmful-request axes — on par with a $7\times$ larger model — and Vyuha’s selective cascade matches an 8B guard’s recall at about a third of its latency on a free T4; and a session-level monitor catches 71% of multi-turn (Crescendo) escalations a per-message moderator catches 0% of. We are explicit throughout about where stronger (white-box, formally verified, or paid-compute) work exists.

1 Introduction

Large language models deployed as assistants and autonomous agents face a moving adversary: jailbreaks that strip safety behaviour, prompt injections hidden in retrieved content and tool metadata, obfuscated attacks, semantic (persuasion) attacks with no surface tell, and responses that leak secrets. Prompt injection is the #1 risk in the OWASP Top 10 for LLM Applications [OWASP, 2025], and Nasr et al. [2025] show that adaptive attackers bypass a dozen recent defenses at $>90\%$ attack-success-rate (ASR) by tuning general optimisers: robustness must be *measured*, not assumed.

Two deployment realities are under-served by current practice. First, guards are typically evaluated on balanced attack/benign sets, whereas production traffic is overwhelmingly benign, so the false-positive rate dominates cost. Second, single-message moderation is the norm, yet multi-turn (Crescendo) attacks [Russinovich et al., 2024] escalate across turns while every individual turn stays benign.

We present **Vyuha**, a layered guardrail built under one operating constraint — it must train and run

on a single free GPU — so that every number is reproducible without a lab budget. Our contributions are: (1) a *deployable* six-layer composition (library, service, CLI) with a selective cascade that keeps the expensive guard off the hot path; (2) an *evaluation methodology* for LLM-guard components — recall-first, with Wilson confidence intervals, an adaptive-attack ablation, a standardised-benchmark reconstruction, and base-rate and contamination discipline; (3) a head-on, end-to-end comparison of the whole pipeline against two 8B guards at matched false-positive rate, in and out of distribution and under a Best-of-N attacker, with ablations that attribute each gain to a layer; (4) three pitfalls in evaluating and composing guards — verdict-position sensitivity, score saturation at strict FPR, and fail-open composition — each found, measured and fixed; and (5) an honest account, at every layer, of where stronger work exists. No single component is a new algorithm; the contribution is the measured composition and its reproducibility.

2 The Vyuha System

Vyuh is a pipeline of six layers (Figure 1), each owning a different failure mode and escalating only when needed (a *selective cascade* that keeps per-request cost low).

L0 — Normalisation. De-obfuscation (Unicode NFKC, Base64/ROT13 decode-and-rescreen, homograph mapping, zero-width/bidi stripping) plus an *adaptive multi-view de-spacing* routine that detects letter-gap width dynamically. This is rule-based, deterministic, and free.

L1 — Fast detector (RJD-v2). An always-on CPU triage (~ 8 ms/prompt): TF-IDF char+word n-grams, engineered features, and a calibrated ensemble, trained with adversarial augmentation. It blocks clear jailbreaks before any LLM runs. It does *not* allow traffic past the guard: as a jailbreak detector it scores 94% of plain harmful requests below its allow threshold (0.20), so letting it skip the guard would wave them through (the skip is an opt-in, off by default).

L2 — Content guard. For harmful-topic and semantic coverage we *compose* an open content guard

(Qwen3Guard-0.6B) that screens every prompt L1 does not block, escalating its most suspicious share to an 8B guard (the selective cascade; Table 4). We do not out-build mature content guards; the stack’s semantic coverage is the composed guard’s.

L3 — Agent defense. An injection scanner de-obfuscates and sanitises untrusted tool outputs; an *MCP tool-poisoning scanner* screens tool *descriptions/metadata* at registration time; a tool policy enforces an instruction hierarchy (system > user > tool), hard-blocking a dangerous action that appears on a tainted turn and was not in the user’s stated intent. This is a free-compute, black-box approximation of capability-based defenses such as CaMeL [Debenedetti et al., 2025], and we say so.

L4 — Output moderation. An egress gate that redacts PII, blocks secret/credential and system-prompt/canary leakage, and scores response harm with the content guard on the (prompt, response) pair.

L5 — Continuous ops. An automated red-team (public attacks only), a population drift monitor, a session-level escalation monitor for Crescendo, and a self-hardening loop.

3 Evaluation Methodology

Deploying LLM-based guards changes what a credible evaluation looks like; we adopt five practices. **(1) Recall-first.** A missed harmful item costs more than a false positive, so recall is the headline metric [Harsh et al., 2026], reported at a fixed low false-positive rate rather than at a default threshold. **(2) Confidence intervals.** Every headline proportion is reported with a Wilson 95% interval, which keeps small- n claims honest (a “perfect” 1.00 on $n=5$ has a wide interval). **(3) Adaptive-attack ablation.** We measure against an attacker that searches over compositions of known evasions — both an exhaustive pairwise sweep and a harder genetic (deep-chain, crossover) search over the same public transforms — and ablate each design choice, rather than reporting a static per-mutator number that understates the attack surface. **(4) Standardised benchmark.** We score the content guard on the 8 NIST-AI-RMF safety categories

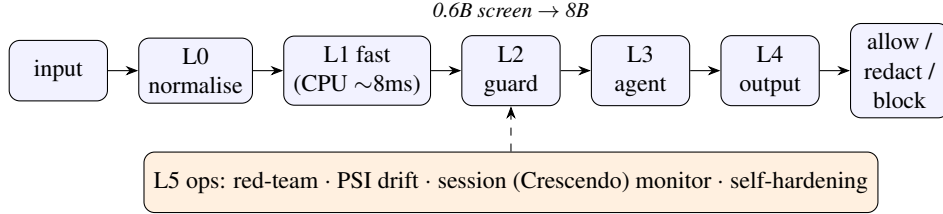


Figure 1: The Vyuha cascade. The CPU fast layer (L1) blocks clear jailbreaks; everything else goes to the L2 guard cascade, where a 0.6B guard screens and only its most suspicious share reaches an 8B guard; L3 defends the agent/tool path; L4 gates egress; and L5 continuously red-teams and monitors.

following Harsh et al. [2026]. **(5) Base-rate and contamination discipline.** Benign controls are drawn from real user prompts; benchmark sets are held test-only. All numbers reproduce from the notebooks, raw scores and analysis scripts in our anonymised artifact (<https://anonymous.4open.science/r/vyuha-ANON>) on a single free T4 GPU (Kaggle or Colab).

4 Results

4.1 The whole pipeline versus a single guard (head-on)

Because every system — Vyuha or a single guard — returns one final decision per prompt, the composition can be compared head-on. We run the same 2,407 prompts through each system: 507 complete harmful requests (NIST-RMF reconstruction), 300 in-the-wild jailbreaks and 1,000 benign prompts held out from L1 training, and 600 *disguised* harmful requests (100 each under Base64, character spacing, homoglyphs, leetspeak, zero-width characters and role-play wrapping). Vyuha is L0 normalisation, then L1, which may block (it is given a quarter of the false-positive budget), then the 0.6B→8B guard cascade on the normalised text with 20% escalation; the baselines are Qwen3Guard-0.6B and Granite Guardian 4.1-8B on the raw text. These rules were fixed before measurement. Every system is calibrated to the same benign false-positive rate on a random half of the benign prompts and evaluated on the other half (500 random splits, positives bootstrapped), so differences are paired and at matched FPR.

Table 1 is the in-distribution result. At matched FPR the whole pipeline catches significantly more

jailbreaks (+0.20 at 5% FPR, +0.23 at 2%) and disguised harmful requests (+0.37, +0.56) than the stronger of the two guards on these pools, is statistically tied with it on plain harmful requests, and runs in about a third of its time. The ablations attribute each gain: without L1 the jailbreak advantage vanishes (0.48→0.19), because guards judge whether content is harmful, not whether a message is trying to subvert the model; without L0 recall on homoglyph and zero-width attacks halves (0.86→0.44), because the guards cannot read the disguised text. A single guard given the same decoding would recover part of the disguised-attack gap (the “without L1” row: 0.84), which is why we report it: the remaining gain, and all of the jailbreak gain, comes from composition. On plain harmful requests the guard does the work in both systems; the quarter of the FPR budget given to L1 is why Vyuha does not exceed Granite there. Two caveats bound the claim: the six disguise tricks are families L1 was augmented against, and the jailbreaks are held out from, but share a source with, L1’s training data. We therefore repeat the comparison out of distribution.

4.2 Out of distribution and under adaptive augmentation

We re-ran the same pre-registered rules on data none of the layers was built or tuned on: 300 WildGuardMix-test *adversarial* harmful prompts, 182 JailbreakBench attack artifacts (PAIR and GCG jailbreaks, Chao et al., 2024), 200 HarmBench requests, and 700 harmful requests under seven obfuscations (five L0 was not designed for — hex, reversed text, double spacing, Unicode tag smuggling, leetspeak+homoglyphs — and two controls). Thresholds are calibrated on a *hard* benign pool of 1,000 prompts that resemble attacks (WildGuardMix adversarial-benign, XSTest-safe, OR-

System	Recall at 5% benign FPR			Recall at 2% benign FPR		
	Harmful	Jailbreaks	Disguised	Harmful	Jailbreaks	Disguised
Qwen3Guard-0.6B alone	0.74	0.18	0.20	0.46	0.08	0.04
Granite Guardian 4.1-8B alone	0.88	0.29	0.56	0.67	0.12	0.31
Vyuha (whole pipeline)	0.87	0.48	0.93	0.78	0.36	0.88
without L1	0.87	0.19	0.84	0.83	0.13	0.78
without L0 (raw text)	0.88	0.52	0.77	0.76	0.36	0.67
Vyuha – Granite 4.1 [95% CI]	−0.01 [−.04,.02]	+0.20 [.11,.32]	+0.37 [.31,.42]	+0.11 [.00,.25]	+0.23 [.12,.33]	+0.56 [.44,.68]

Table 1: Head-on, end-to-end comparison at matched benign FPR (held-out FPR 5.3–5.4% and 2.1–2.2% for every system). Vyuha’s mean latency is 203 ms per request vs 595 ms for Granite 4.1 on one T4 (95% benign / 5% attack traffic). Bold: significant difference from Granite 4.1 alone (paired, 500 splits). Disguised = six obfuscation tricks; Granite 4.1 alone catches 0.01 of Base64-encoded requests, Vyuha 1.00.

Bench-hard). We add Llama Guard 3-8B as a second strong baseline, and an adaptive black-box attacker: Best-of-N augmentation [Hughes et al., 2024] (random capitalisation, word scrambling, character noise; 20 variants for each of 60 HarmBench requests; a request evades if any variant is allowed — an upper bound on attack success, since we do not check that the target still understands it).

Table 2 narrows the headline. **Out of distribution, Vyuha’s jailbreak advantage does not transfer:** on adversarial WildGuardMix prompts and on the JailbreakBench PAIR/GCG artifacts it is tied with Granite 4.1 (0.35 vs 0.34 on JailbreakBench) and trails Llama Guard 3 (0.60, significantly), which is also significantly better on plain HarmBench requests. L1 adds nothing on these new jailbreaks (it fires on almost none of them) and only consumes a quarter of the budget: without L1, Vyuha reaches 0.40 on JailbreakBench. The strongest single guard therefore depends on the data, and our earlier “strongest single guard” was strongest only on our own pools. **The gain that does transfer is on encoded and obfuscated input:** +0.65 over Granite 4.1 and +0.61 over Llama Guard 3 at 2% FPR, at a third of the latency. L1 contributes most of it (without L1: 0.69→0.35); both 8B guards catch essentially none of the hex, reversed or tag-smuggled requests. Two misses are ours: reversed text (0.03) and Unicode tag smuggling (0.00) — L0 strips tag characters as invisible instead of decoding them, deleting the hidden payload before any guard sees it, a fail-open bug this run exposed (now fixed: L0 decodes tag characters into a separate view; the numbers reported are from before the fix). Under Best-of-N augmentation every

system except Llama Guard 3 is evaded for all 60 requests within five variants; Vyuha inherits the weakness of its 0.6B screen, because a variant the screen scores low never reaches the expert. In an exploratory, post-hoc run (not pre-registered) replacing Granite 4.1 with Llama Guard 3 as the cascade expert recovers Llama Guard 3’s adversarial, JailbreakBench (0.56 vs 0.60) and plain-harmful recall within the confidence interval while keeping the obfuscation gain (+0.55), but still not its Best-of-N robustness, which the screen gates away. The remaining subsections report each layer’s own evidence.

4.3 Layer-level evidence

Table 3 summarises the measured results; all proportions carry 95% confidence intervals.

Obfuscation is closed cheaply (C1). RJD-v2 reaches ROC-AUC 0.923, recall@1%FPR 0.330, over-refusal 0.044 at ~8 ms on CPU — matching a GPU DeBERTa injection guard (0.896 / 0.113) at ~8× lower latency and lower over-refusal. Recall under attack is 1.00 on Base64, ROT13, character-spacing and a *held-out* wider-spacing variant; the adaptive de-spacing fix closed the character-spacing gap from ASR 0.83 to 0.00 and generalises to the held-out variant.

Adaptive robustness (C4). Figure 2 shows the ablation. Against an adaptive attacker (exhaustive search over 101 single/paired evasions per seed) on 150 real in-the-wild jailbreak seeds: a bare detector is fully evaded (1.00). Normalisation alone drops the *static* per-mutator ASR to 0.02 yet the adaptive attacker still

System	At 2% benign FPR (hard benign pool)					At 5% benign FPR (hard benign pool)				
	Adversarial	JBB	HarmBench	Unseen obf.	BoN evasion _L	Adversarial	JBB	HarmBench	Unseen obf.	BoN evasion _L
Qwen3Guard-0.6B alone	0.12	0.10	0.62	0.01	1.00	0.26	0.33	0.78	0.04	1.00
Granite Guardian 4.1-8B alone	0.08	0.34	0.83	0.05	1.00	0.14	0.47	0.87	0.06	1.00
Llama Guard 3-8B alone	0.16	0.60	0.89	0.08	0.90	0.30	0.75	0.94	0.24	0.51
Vyuha (whole pipeline)	0.10	0.35	0.82	0.69	1.00	0.18	0.48	0.85	0.73	1.00
without L1	0.08	0.40	0.83	0.35	1.00	0.14	0.57	0.86	0.40	1.00
without L0 (raw text)	0.12	0.36	0.82	0.58	1.00	0.20	0.48	0.86	0.62	1.00
Vyuha — Granite 4.1 [95% CI]	+0.02 [−.03,.08]	+0.01 [−.07,.10]	−0.02 [−.05,.03]	+0.65 [.61,.69]	0.00	+0.04 [−.01,.09]	+0.01 [−.06,.11]	−0.02 [−.05,.02]	+0.67 [.62,.72]	0.00
Vyuha — Llama Guard 3	−0.06 [−.14,.02]	−0.25 [−.38,−.12]	−0.07 [−.14,−.01]	+0.62 [.56,.67]	+0.10	−0.12 [−.21,−.03]	−0.27 [−.39,−.17]	−0.09 [−.15,−.04]	+0.49 [.37,.58]	+0.49

Table 2: Out-of-distribution head-on comparison (same pre-registered rules as Table 1; 500 splits). BoN evasion = share of 60 requests with at least one of 5 Best-of-N variants allowed (lower is better). JBB = JailbreakBench PAIR/GCG artifacts. Bold in difference rows: 95% interval excludes 0. Vyuha’s latency is 192 ms vs 595 ms.

Axis (layer)	Metric	Result
Obfuscation (L1)	recall	1.00 incl. held-out
	latency	~8 ms CPU
Adaptive (L1)	ASR, L0 only	1.00 [0.97,1.00]
	ASR, L0+aug (pairwise)	0.03 [0.01,0.08]
	ASR, L0+aug (genetic)	0.08 [0.06,0.10]
	held-out seeds (gen.)	0.30 [0.23,0.38]
Self-harden (L5)	seen-family ASR	0.11 → 0.00
Semantic (L2)	PAIR recall	0.90 (L1: 0.07)
NIST-RMF (L2)	recall, req. axes	0.886 (≈4B: 0.84)
	benign FPR	0.071
Agent inj. (L3)	AgentDojo ASR	0.39 → 0.33
MCP poison (L3)	detection	1.00 [0.87,1.00]
Multi-turn (L5)	session detect	0.71 [0.65,0.77]
	per-message	0.00
Output (L4)	leak/harm F1	1.00

Table 3: Vyuha, security-grade metrics with 95% CIs (genetic ASR pooled over five attacker seeds). Every value reproduces from the anonymised artifact on a single free GPU.

reaches 1.00 [0.97,1.00] — a +0.98 “attacker-moves-second” premium the static number hides. Adversarial augmentation is the load-bearing fix: the full detector holds adaptive ASR to 0.03 [0.01,0.08] at 1% benign FPR. A harder *genetic* attacker (deep transform chains with crossover, ~110 queries/seed), run with five independent attacker seeds, confirms this rather than breaking it — the full detector holds at 0.08 [0.06,0.10] pooled (0.07–0.09 per seed), and even an attacker allowed all five seeds evades only 0.09 of prompts. The same genetic search, however, drives *augmentation-alone* from 0.25 to 0.75 [0.72,0.78] pooled — highly seed-dependent (0.58–0.99), and 1.00 for the five-seed attacker: L0 normalisation is far more load-bearing than the pairwise ablation implied, and neither component is sufficient on its own — both

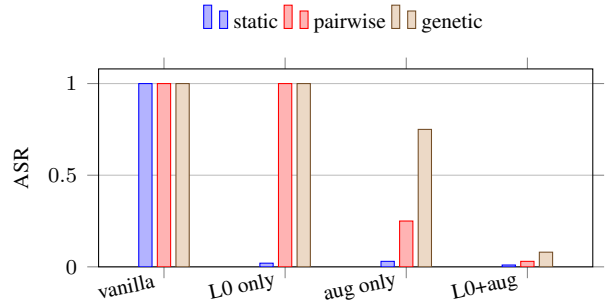


Figure 2: Adaptive-attack ablation (C4), 150 in-the-wild seeds. L0 alone cuts *static* ASR to 0.02 but both adaptive attackers still reach 1.00. Only the full detector (L0+aug) holds — 0.03 [0.01,0.08] pairwise and 0.08 [0.06,0.10] under a harder genetic search (pooled over five attacker seeds) — while augmentation *alone* collapses to 0.75 under the genetic attacker (1.00 for an attacker allowed all five seeds), showing L0 and augmentation are both load-bearing.

earn their place.

Held-out seeds and self-hardening (L5). The adaptive seeds above come from the detector’s training distribution. On a disjoint split — 150 attack seeds and 500 benign prompts removed from training, against a detector retrained without them — the genetic attacker evades 0.30 [0.23,0.38] of *held-out* seeds versus 0.11 [0.07,0.17] of seen ones: robustness to attacks from unseen families is markedly weaker, which we report as a limit of the fast layer. L5’s self-hardening loop addresses the seen side. Each round it harvests the variants that evaded, adds normalised signatures, retrain, and rolls the round back if over-refusal on a 500-prompt benign guard set rises by more than 2 points; each round faces a *fresh* attacker seed, and a no-

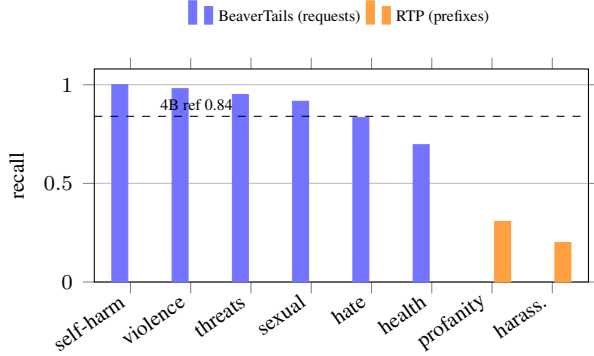


Figure 3: NIST-RMF per-category recall (Qwen3Guard-0.6B). On complete-harmful-request axes (BeaverTails) it meets or exceeds the benchmark’s 4B leader (0.84, dashed); the two toxicity-prefix axes (RealToxicityPrompts) are short fragments a request-focused guard flags less.

hardening control runs the same rounds. One round drives seen-family ASR to 0.00 [0.00,0.02] (control 0.13 [0.08,0.19]) with no change in over-refusal on held-out benign prompts (6.6% $\rightarrow$ 6.4%); held-out ASR does not move significantly (0.27 [0.21,0.35] vs. control 0.30 [0.23,0.38]). Self-hardening therefore closes attack families once they have been seen — at no measured usability cost — but does not generalise to novel ones, which remain the content guard’s and the red-team loop’s job.

Division of labour (L1 vs L2). On 103 real PAIR jailbreaks the surface detector flags 6.8% (no surface tell) while the composed content guard flags 90.3% at 4.8% over-refusal — semantics are, by design, L2’s job, not L1’s.

NIST-AI-RMF guard benchmark (L2). Figure 3 shows the per-category split. Scoring the 0.6B content guard on the 8 NIST-AI-RMF categories [Harsh et al., 2026] (a reconstruction from BeaverTails and RealToxicityPrompts; 807 unsafe / 800 benign; not the official split): scoring the guard’s continuous $P(\text{unsafe})$, recall is 0.886 on the six complete-harmful-request axes at 7.1% benign FPR — on par with the benchmark’s best model (Qwen Guard 4B, 0.840) despite being $\sim 7\times$ smaller — and 0.253 on the two toxicity-prefix axes (short fragments a request-focused guard flags less), for an aggregate 0.651 [0.62,0.68]. A

small guard is competitive with a much larger one on complete harmful requests; the per-category spread is why L2 composes guards. Composition must be *calibrated*: each member is thresholded on its continuous score so that the union meets a fixed total benign FPR (split equally between members, fixed before measurement), never a raw OR of hard verdicts, which inherits the loosest member’s false-positive rate. Its measured value is reported with the baselines below.

Cross-family guard comparison (matched FPR).

Table 4 compares the guard Vyuha binds at L2 — the off-the-shelf Qwen3Guard-Gen-0.6B; Vyuha’s contribution is how it is composed, not the guard itself — with five current open guards from four families (0.6B–8B), all scored identically on the six complete-harmful-request axes at matched benign FPR. Each guard is scored by the raw log-odds of its verdict tokens, read at the guard’s *own* verdict position (after Qwen3Guard’s `Safety:` prefix, Granite Guardian 4.1’s empty `<think>` block and `<score>` tag, Llama Guard’s leading newlines), and a row is admitted only if the sign of its score agrees with the guard’s own generated verdict on $\geq 90\%$ of a 100-prompt check (observed 94–100%). We rank on log-odds rather than sigmoid probabilities because saturated probabilities tie at the benign quantile and silently zero recall at strict FPR — a pitfall we hit and corrected, and recommend guarding against in any matched-FPR comparison. Precision is matched: guards that fit a 16 GB T4 in fp16 are compared in fp16, the 4B–8B guards in 4-bit against a 4-bit run of ours. Differences are tested with a paired, stratified bootstrap ($B=2000$) that re-estimates the FPR thresholds on every draw, so its intervals cover threshold noise as well as sampling noise.

At matched precision, the 0.6B guard *significantly* beats ShieldGemma-2B on all three metrics (AUC +0.09, recall +0.25 at 2% and +0.32 at 5% FPR) and the $13\times$ -larger Llama Guard 3 on AUC (+0.19) and recall at 5% FPR (+0.10), tying it at 2%. It is statistically indistinguishable from Granite Guardian 3.2 (3B, 0.8B active) in fp16 on all three metrics; in 4-bit, Granite 3.2 is significantly better at 2% FPR (−0.13). Granite Guardian 4.1-8B and the 4B Qwen3Guard are significantly better on every metric. Four-bit quantisation itself costs the 0.6B guard little but not nothing (AUC −0.006, recall −0.06 at 5% FPR), so the 4-bit

8B rows, if anything, slightly understate those guards. Per axis (5% FPR, matched precision, same bootstrap), the 0.6B guard’s advantage over ShieldGemma and Llama Guard 3 comes from hate speech (+0.57, +0.22) and sexual content (+0.66, +0.34), with no significant difference on the other axes; against Granite 3.2 it differs significantly only on violence (−0.07); Granite 4.1 and the 4B Qwen3Guard lead on every axis except self-harm, whose $n=15$ is too small to separate any pair. The 0.6B guard’s weakest axes are hate speech (0.61) and health misinformation (0.54) — exactly where the cascade’s escalation target is strongest (0.85, 0.74). Held out, the thresholds transfer: fit on a random 400 of the benign prompts and evaluated on the other 400 (500 splits), every guard’s recall is unchanged to within 0.015 and the achieved FPR averages 2.1–2.2% / 5.1–5.2% against 2% / 5% targets; the cascade’s parity with Granite 4.1 also holds (2% FPR, 20% escalation: 0.85 vs 0.80).

Selective cascade. Because the strongest guard is $13\times$ larger, Vyuha’s L2 cascade lets the 0.6B guard screen all traffic and escalates only its most suspicious share to the 8B guard, which decides; thresholds are set so the *total* benign FPR matches the target. With escalation fixed in advance at 10/20/30% of benign traffic (all reported), the cascade matches Granite 4.1-8B alone with no significant difference at 20% and 30% escalation (2% FPR: 0.85 and 0.84 vs 0.80; 5% FPR: 0.87 and 0.90 vs 0.90), i.e. 8B-level recall while invoking the 8B guard on 70–80% fewer benign requests; at 10% escalation it is significantly worse at 5% FPR (−0.09), so the escalation budget matters. In latency on a single T4 (batch 1, mean over 200 benchmark prompts), the 0.6B guard scores a prompt in 66 ms and Granite 4.1 in 595 ms; with 20% escalation and 1–5% harmful traffic the cascade averages 189–206 ms per request — 8B-level recall at about a third of the 8B guard’s latency, with both models resident in under 8 GB of GPU memory. By contrast, an OR-union of the two guards at a split FPR budget is *worse* than Granite 4.1 alone (0.73 vs 0.80 at 2% FPR) because each member must run at a stricter threshold; unions help only between comparable members (ours OR Granite 3.2: 0.62, at least either alone). The operational rule: cascade to the strongest affordable guard; compose only peers.

Agent and multi-turn defense (L3, L5). On Agent-Dojo (all four suites, *important_instructions*, gpt-oss-120b agent; injection ASR is the share of episodes in which the injection goal was executed, with errored episodes counted as successful attacks; 949 episodes per arm), L3 lowers injection ASR from 0.39 [0.35,0.42] to 0.33 [0.30,0.36] ($p=0.015$) with no utility cost (0.50 vs. 0.52). The gain is confined to two suites — Slack (−0.15, $p=0.02$) and Travel (−0.16, $p=0.005$) — with no significant change in Banking or Workspace: a modest, real reduction, far from the capability-based guarantees of CaMeL [Debenedetti et al., 2025]; injection-under-obfuscation detection is 1.00 versus 0.00–0.17 for a regex baseline. The MCP scanner detects 1.00 [0.87,1.00] of poisoned tool metadata at zero false positives on 25 poisoned / 27 benign tools. The session-level monitor flags 71% [0.65,0.77] of modeled Crescendo escalations at 0% benign FPR, versus 0% for a per-message moderator (every turn stays sub-threshold).

External head-to-head, and a detection-vs-mitigation gap (PIArena). We ship Vyuha as a PIArena [Geng et al., 2026] defense plug-in and evaluate it on *squad_v2* under the *combined* attack, targeting an open Qwen-3-4B application and scored by an *independent* judge (a GPT-OSS-120B model, distinct from the target’s family; $n=200$).¹ This surfaces an honest, instructive result. First, the surface detector (L1) does *not* fire on this attack distribution (0% at its operating threshold) — these are out-of-distribution from its in-the-wild jailbreak training — so the selective cascade would not engage on L1 alone. The layer scoped to instructions-injected-into-content, L3’s injection scanner, instead **detects 1.00 [0.98,1.00] of the injected contexts at 0.5% benign FPR**: defense-in-depth covering L1’s blind spot, exactly as designed. *Mitigation*, however, is the hard part: neutralising the injected instruction while preserving the ability to answer the legitimate question. Both strategies we tried reduce attack success but at a steep utility cost —

¹*Measurement validity.* A minority of attacked responses are short error-like strings (e.g. “Server Error 500”, “Access denied”). A per-sample diagnostic confirms these are *attack-induced* — the *combined* attack’s injected instruction directs the model to emit exactly that text — and are not backend or infrastructure failures, so they are correctly scored as attack successes and do not spuriously shift ASR in either direction.

Guard	Precision	Params	AUC	R@2%FPR	R@5%FPR	ms
Qwen3Guard-Gen-0.6B (Vyuha L2)	fp16	0.6B	0.92	0.54 [0.49,0.58]	0.73 [0.69,0.77]	66
ShieldGemma	fp16	2B	0.83*	0.29 [0.25,0.33]*	0.41 [0.37,0.45]*	117
Granite Guardian 3.2	fp16	3B [†]	0.91	0.64 [0.60,0.68]	0.77 [0.73,0.81]	162
Qwen3Guard-Gen-0.6B (Vyuha L2)	4-bit	0.6B	0.91	0.49 [0.44,0.53]	0.68 [0.64,0.72]	107
Llama Guard 3	4-bit	8B	0.72*	0.55 [0.50,0.59]	0.58 [0.53,0.62]*	593
Qwen3Guard-Gen (L2 slot)	4-bit	4B	0.94*	0.81 [0.78,0.84]*	0.86 [0.83,0.89]*	440
Granite Guardian 4.1	4-bit	8B	0.95*	0.80 [0.76,0.83]*	0.90 [0.87,0.92]*	595
<i>Vyuha selective cascade: 0.6B screens all traffic, escalates a pre-set share of benign traffic to Granite 4.1</i>						
escalate 20%	fp16+4-bit	—	—	0.85 [0.82,0.88]	0.87 [0.84,0.90]	189
escalate 30%	fp16+4-bit	—	—	0.84 [0.81,0.87]	0.90 [0.87,0.92]	248

Table 4: Matched-FPR guard comparison on the six complete-harmful-request axes (NIST-AI-RMF reconstruction, $n=507$ unsafe / 800 benign; Wilson 95% CIs on recall). Scores are raw verdict-token log-odds at each guard’s own verdict position, each validated against the guard’s generated verdict (94–100% agreement). Bold: best per block. *Significantly different from the 0.6B guard at the same precision (paired stratified bootstrap, $B=2000$, 95%); unmarked differences are not significant. Neither cascade row differs significantly from Granite 4.1 alone. ms: mean latency per prompt on one T4 at batch size 1 (200 benchmark prompts); cascade rows are expected per-request latency at 1% harmful traffic. [†]Mixture-of-experts, 0.8B active.

surgical sentence-removal takes ASR $0.905 \rightarrow 0.630$ but utility $0.535 \rightarrow 0.210$, and provenance spotlighting takes ASR to 0.485 but utility to 0.154 (worse, as the model over-generalises the “treat as data” instruction and under-answers). Both drop utility *below* the no-defence baseline, so neither is a deployable auto-repair. The honest, deployable reading is that L3’s value here is **high-precision detection** — block or escalate the flagged request (accurate, and utility-safe on the 99.5% of benign traffic) — rather than silently sanitising and answering a poisoned context. Utility-preserving mitigation of injected-context attacks is an open problem we quantify rather than paper over (Limitations).

Across target backends. The detection–mitigation gap is not specific to one target. Table 5 repeats the head-to-head on a different-family backend (Llama-3.1-8B); L3 detection is input-side and backend-independent (1.00 [0.98,1.00]), so the table reports the *attack and mitigation* outcome under the default (spotlight) configuration. The combined attack lands differently per backend — Qwen-3-4B complies with the injected task far more (ASR 0.92 vs 0.42) while still answering, whereas Llama-3.1-8B resists the injected task but its answers are more often suppressed by it (utility 0.18 vs 0.53) — yet on *both*, spotlight mitigation

Backend	Metric	none [95% CI]	Vyuha [95% CI]
Qwen-3-4B	ASR	0.92 [0.87,0.95]	0.54 [0.47,0.61]
	Utility	0.53 [0.46,0.60]	0.18 [0.13,0.24]
Llama-3.1-8B	ASR	0.42 [0.36,0.49]	0.42 [0.35,0.48]
	Utility	0.18 [0.13,0.23]	0.17 [0.12,0.23]

Table 5: PIArena head-to-head across target backends (*combined* attack, independent GPT-OSS-120B judge, $n=200$ each, Wilson 95% CIs); Vyuha at its default spotlight config. L3 detection is backend-independent (1.00 [0.98,1.00]). The fresh 4B spotlight figures agree within CIs with the per-strategy run reported above; on neither backend does mitigation improve the ASR/utility trade-off — the null generalisation that motivates deploying L3 as a detection gate.

fails to improve the trade-off, because L1 is out-of-distribution and the cascade does not engage. This confirms, across backends, that L3’s deployable value is high-precision *detection* (block/escalate), not in-place repair.

4.4 Three pitfalls in evaluating and composing guards

Building and measuring Vyuha surfaced three failure modes that silently change conclusions. Each was

found by a validity check, fixed, and pinned by a regression test. None is new in kind — related effects are reported for guard strictness [FlexGuard, 2026], score ties [Rank compression, 2026] and confident guard misses [Hong et al., 2026] — but we document their size on current guards because each would mislead a practitioner who composes or benchmarks them.

(1) Verdict-position sensitivity. A guard’s score is only meaningful if read at the token where that guard actually states its verdict. Granite Guardian 4.1 emits an empty reasoning block and a `<score>` tag first, Llama Guard leading newlines, Qwen3Guard a `Safety:` prefix. Reading the first generated token instead gave Granite 4.1 an AUC of 0.19 (worse than chance) and Llama Guard 3 zero recall at strict FPR; reading each guard at its own verdict position, admitted only after $\geq 90\%$ agreement with its own generated verdict, restored AUC 0.95 and 0.72. Benchmarks that score many guards with one generic reader can therefore misrank them.

(2) Score saturation at strict FPR. Converting verdict logits to probabilities saturates many benign scores at exactly 1.0; the benign quantile that defines a 2% FPR threshold then ties at the maximum and no attack can exceed it. Our own 0.6B guard’s recall at 2% FPR read 0.27 under probabilities and 0.54 under raw log-odds on the same outputs (cf. rank compression of verdict probabilities [Rank compression, 2026]). Matched-FPR comparisons should rank on log-odds and report the achieved FPR and the fraction of ties at the threshold.

(3) Fail-open composition. Related to the confident-miscalibration failure of CPU cascade stages [GuardChain, 2025], but arising from a *task mismatch* rather than distribution shift: layered guardrails commonly let a cheap first layer *allow* low-risk traffic so the expensive guard is spared. Our L1 is a jailbreak detector: it scores 94% of plain harmful requests below its allow threshold, so an “allow below 0.20” shortcut would have passed them without any content check — a composed system strictly weaker than its own guard. Vyuha now lets L1 only *block*; skipping the guard is opt-in. The general rule: a layer may decide on

another layer’s behalf only for the attack class it was calibrated on.

5 Deployment and Governance

Vyuha ships as a Python library, a FastAPI service (`/scan`, `/moderate`, `/guard_turn`), and a CLI, so a team can place it in front of or behind an LLM with a few lines of code.

Cost. We measure cost as compute, not tokens. Counting each guard’s exact input (its template plus the prompt) on the 1,607 benchmark prompts, the cascade processes *more* tokens than a single 8B guard (561 vs 320 per request: the 0.6B guard’s template is long and it reads every request) but needs about a third of the compute (1.9 vs 5.4 TFLOPs per request, ≈ 2 FLOPs per parameter per token), consistent with its measured latency (189 vs 595 ms). The lever a deployment controls is the escalation share, not the token count. Because production traffic is overwhelmingly benign, thresholds are set at a fixed low false-positive rate, where a small change in FPR dominates total operating cost; recall is then reported at that operating point rather than at a default 0.5.

Governance. Layers map onto the OWASP LLM Top 10 and the NIST AI RMF Measure/Manage functions (Table 6); risks outside scope are named rather than papered over. Continuous red-teaming, the PSI drift monitor, and the self-hardening loop support lifecycle maintenance as the attack mix shifts — the operational-maturity concern this track foregrounds — and every reported number is regenerable from the versioned notebooks and committed raw scores in the artifact, giving a simple audit trail.

6 Related Work

Detectors and guards. Deployable prompt-injection detectors such as PromptShield [Jacob et al., 2025] fine-tune a single classifier for a low-false-positive regime; open safety guards (Qwen3Guard, Granite Guardian, Llama Guard, WildGuard, ShieldGemma) each own different axes, and no one model dominates [Harsh et al., 2026]. Vyuha does not out-build any of these — it *composes* them behind a CPU fast

OWASP LLM risk	Covered by
LLM01 Prompt injection	L0–L3
LLM02 Sensitive-info disclosure	L4 PII/secret redaction
LLM06 Excessive agency	L3 tool policy + Dual-LLM
LLM07 System-prompt leakage	L4 canary + n-gram
LLM09 Misinformation	L2/L4 response scoring
<i>Out of scope, stated explicitly:</i> LLM03 supply chain, LLM04 data poisoning, LLM10 unbounded consumption.	

Table 6: Governance mapping. Also aligned to NIST AI RMF (Measure/Manage) and MITRE ATLAS.

layer and measures the resulting division of labour. Closest to our cascade, COGNIT-Guard [COGNIT-Guard, 2026] couples a calibrated CPU gatekeeper with confidence-gated escalation to a larger guard under explicit FPR and latency constraints, and GuardChain [GuardChain, 2025] routes prompts $\text{Regex} \rightarrow \text{CPU} \rightarrow \text{GPU}$ and reports that CPU classifiers fail out of distribution through *confident* false negatives that never escalate; and Alotaibi et al. [2026] show that the layers of a defence stack fail in correlated ways, so a stack must be measured end to end — their seven-layer stack was indistinguishable from its strongest layer while refusing most benign prompts. Our head-on results (Tables 1, 2) are such end-to-end measurements at matched FPR: the composition beats its strongest layer on encoded and disguised text, in and out of distribution, but its jailbreak advantage is in-distribution only. Cascades are also attackable as systems: staged attacks break safeguard pipelines component by component [STACK, 2025], and adversarial suffixes can steer an LLM cascade’s escalation decisions [Sun et al., 2026]. Encoded and character-level evasions of guards are well documented [Hackett et al., 2025, Wei et al., 2025], and semantic censorship of invertible encodings is impossible in general [Glukhov et al., 2024]; L0 therefore narrows, and cannot close, this gap. PIDS-Bench [PIDS-Bench, 2026] finds that high-F1 injection detectors still leave substantial attack success, consistent with our L3 result on AgentDojo. **Adaptive attacks and hardening.** Static defenses fall to adaptive attackers: Nasr et al. [2025] bypass twelve defenses at $>90\%$ ASR, and PISmith [Yin et al., 2026] trains an RL red-teamer, on the PIArena platform [Geng et al., 2026], that beats state-of-the-art prompt-injection defenses. Our adaptive-attack ablation is a black-box, search-based instance of this

discipline; a learned RL attacker is stronger than ours and a natural stress test. We therefore ship Vyuha as a PIArena defense plug-in (released with the repository), so it is measured by the same static and search-based suite (Direct, Combined, TAP, PAIR) alongside PIArena’s published defenses; the RL attacker is a multi-GPU upper bound we cite rather than claim to withstand (see Limitations). HASTE [HASTE, 2026] proactively hardens detectors by generating evasive prompts; our L5 self-hardening loop is a lightweight, free-compute, signature-based counterpart that we do not claim to improve upon. **Agentic and capability defenses.** CaMeL [Debenedetti et al., 2025] gives formal capability guarantees our black-box L3 only approximates; representation-level defenses report lower residual ASR but require white-box model access.

Our position. Against this landscape Vyuha’s contribution is not a new detector or attacker but the *measured composition* — a complete L0–L5 stack shipped as a library, service and CLI — its *reproducibility on a single free GPU*, and an *honest evaluation methodology* (recall-first, confidence intervals, an adaptive ablation, and a standardised benchmark) applied to the *deployed system* rather than to one component in isolation. Crucially, the composed guards are an interchangeable slot, not the contribution: L2 currently binds the strongest small open guards current at writing (Qwen3Guard-0.6B, with Granite Guardian 4.1-8B and Llama Guard 4 as complementary members — all 2026 releases and, as of this writing, the latest of their respective families), but the cascade, the selective-invocation policy, the fixed-FPR calibration, and the measurement harness are model-agnostic and hot-swap whatever guard leads next — including 2026 entrants beyond these families. The delta therefore does not decay when a newer guard ships — it is the harness and the methodology, which the individual detectors of Jacob et al. [2025], Yin et al. [2026] and HASTE [2026] do not provide.

7 Conclusion

A credible, modern LLM guardrail can be assembled from independent, individually cheap layers entirely on free compute, and — crucially — measured honestly, with confidence intervals, adaptive attacks, and

a standardised benchmark, so that its real strengths and limits are visible rather than asserted. Measured as a whole and out of distribution, the composition’s durable advantage is narrow but large: encoded and obfuscated requests that 8B guards cannot read, at a third of their cost. Its in-distribution jailbreak advantage did not transfer, and it does not resist Best-of-N augmentation — the guard behind the cascade matters, and the cheap screen in front of it gates away the expert’s robustness.

Limitations

The in-distribution comparison (Table 1) overstates the jailbreak gain, which did not survive out of distribution (Table 2); the out-of-distribution run uses one augmentation attacker, and measures evasion rather than target-model harm; all systems remain weak on jailbreaks in absolute terms (at most 0.48 recall at 5% FPR), and the 2% FPR estimates have wide intervals (500 benign prompts per calibration half). The surface detector (L1) does not and should not catch harmful-topic or semantic attacks; those depend entirely on the composed content guard, whose semantic ceiling we do not improve upon. Our own fine-tuned guard is jailbreak-only. The agent layer (L3) is a lightweight black-box approximation without the provenance guarantees of capability-based defenses; AgentDojo is run with a single agent model, and a same-backend comparison to CaMeL’s published mitigation is left to future work. The NIST-RMF result is an indicative reconstruction, not the official 79,331-sample split, and the two toxicity-prefix axes are short fragments a request-focused guard reasonably under-flags. The Crescendo result is on modeled score trajectories. Our adaptive attacker searches over public transforms — both an exhaustive pairwise sweep and a harder genetic (deep-chain, crossover) search, which the full detector also survives (0.08 [0.06,0.10] genetic over five seeds vs 0.03 pairwise). Those seeds are in-distribution; on held-out attack seeds the genetic attacker evades 0.30 [0.23,0.38], and self-hardening closes seen attack families but not novel ones. To place Vyuha in a shared harness we release it as a PIArena [Geng et al., 2026] defense plug-in; the static and search-based head-to-head (Direct, Combined, TAP, PAIR) is single-GPU runnable, whereas

the strongest cited attacker — the *learned* RL red-teamer PISmith [Yin et al., 2026], which optimises against the model rather than composing fixed transforms — requires a multi-GPU training rig, so that comparison is a stated upper bound rather than a claim of robustness. **Auto-mitigation of injected-context attacks is an open problem we quantify rather than solve:** on the PIArena head-to-head L3 *detects* the injection at 1.00 [0.98,1.00] (0.5% FPR), but both mitigations we tried (sentence-removal and provenance spotlighting) reduce ASR only by trading away utility below the no-defence baseline — so we recommend deploying L3 as a high-precision block/escalate gate, and leave utility-preserving mitigation (e.g. dual-LLM quarantine) to future work. Our external head-to-head is measured on two target backends (Qwen-3-4B and Llama-3.1-8B) at $n=200$ each; still-larger backends and prompt sets are future work. The cross-family guard comparison is on one benchmark reconstruction (BeaverTails/RealToxicityPrompts prompts, $n=507/800$). With 400 benign prompts per calibration half, the achieved FPR at a 2% target varies from 0.5% to 4.8% across splits, so deployments should calibrate on far more benign traffic. BeaverTails prompts derive from HH-RLHF red-team data, which some guards (e.g. Granite Guardian) report training on, so absolute recalls may be optimistic for any guard. The 8B guards run in 4-bit on a T4; Llama Guard 4 could not be loaded on our stack and is not included. Some efficiency and attack-cost figures are estimates under a measured detection rate, labelled as such. No stack is unbreakable; Vyuha raises cost, it does not guarantee safety.

Ethics

Vyuha is a defensive safety filter. Its red-team and attack code mutate only attacks already present in public corpora, for evaluation and hardening; the repository contains no novel weaponisation. Harmful-content datasets are loaded read-only for measurement and no harmful text is authored. PII redaction is best-effort, not a compliance guarantee. The system is intended to operate with human oversight and continuous red-teaming.

References

- OWASP Foundation. 2025. *OWASP Top 10 for Large Language Model Applications (2025)*. <https://genai.owasp.org>.
- Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander Schulhoff, Jamie Hayes, Ilia Shumailov, Andreas Terzis, and Florian Tramèr. 2025. The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections. *arXiv:2510.09023*.
- Mark Russinovich, Ahmed Salem, and Ronen Eldan. 2024. Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack. *arXiv:2404.01833* (USENIX Security 2025).
- Reetu Raj Harsh, Bhaskarjit Sarmah, and Stefano Pasquali. 2026. Benchmarking Open-Source Safety Guard Models: A Comprehensive Evaluation. *arXiv:2605.28830* (ICLR 2026 Workshop).
- Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. *arXiv:2503.18813*.
- Dennis Jacob et al. 2025. PromptShield: Deployable Detection for Prompt Injection Attacks. *arXiv:2501.15145*; ACM CODASPY 2025.
- Chenlong Yin, Runpeng Geng, Yanting Wang, and Jinyuan Jia. 2026. PISmith: Reinforcement Learning-based Red Teaming for Prompt Injection Defenses. *arXiv:2603.13026* (COLM 2026).
- Runpeng Geng, Chenlong Yin, Yanting Wang, Ying Chen, and Jinyuan Jia. 2026. PIArena: A Platform for Prompt Injection Evaluation. *arXiv:2604.08499* (ACL 2026).
- Proactive Hardening of LLM Defenses with HASTE. 2026. *arXiv:2601.19051*; NDSS.
- Do You Really Need a GPU to Guard Your LLM? CPU-Class Classifiers and Multi-Stage Pipelines for Safety Enforcement at Scale. 2025. *arXiv:2512.19011*.
- Abra Alotaibi, Muhammad Shahid Jabbar, Saddam Al-Azani, and Moataz Ahmed. 2026. Layered LLM Defenses as an Ensemble: Access Tiers, Inference Cost, and the Measured Failure Correlation Between Defense Layers. *arXiv:2608.28327*.
- STACK: Adversarial Attacks on LLM Safeguard Pipelines. 2025. *arXiv:2506.24068*.
- Zehan Sun, Dingfan Chen, and Songze Li. 2026. When Efficiency Backfires: Cascading LLMs Trigger Cascade Failure under Adversarial Attack. *arXiv:2605.17288*.
- COGNIT-Guard: Calibrated Standalone Direct-Decision Guardrails with Heterogeneous CPU-NPU Confidence Cascading under Explicit Latency and False-Positive Constraints. 2026. *arXiv:2609.33671*.
- Jonghyun Hong, MinJae Jung, and Minwoo Kim. 2026. Guard Models Are Overconfident Where Base Models Are Uncertain. *arXiv:2609.36477* (Findings of EMNLP 2026).
- A VLM Answer Is Not an Anomaly Score: Rank Compression Across Image and Video Anomaly Detection. 2026. *arXiv:2608.21244*.
- FlexGuard: Continuous Risk Scoring for Strictness-Adaptive LLM Content Moderation. 2026. *arXiv:2602.23636* (ACL 2026).
- PIDS-Bench: Evaluating Prompt-Injection Detectors Under Over-Defense, Obfuscation, and Distribution Shift. 2026. *arXiv:2609.15017*.
- William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. 2025. Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. In *Proc. LLMSEC Workshop*. *arXiv:2504.11168*.
- Emoji Attack: Enhancing Jailbreak Attacks Against Judge LLM Detection. 2025. *arXiv:2411.01077* (ICML 2025).
- Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed

Hassani, and Eric Wong. 2024. JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models. In *NeurIPS Datasets and Benchmarks*.

John Hughes, Sara Price, Aengus Lynch, Rylan Schaeffer, Fazl Barez, Sanmi Koyejo, Henry Sleight, Erik Jones, Ethan Perez, and Mrinank Sharma. 2024. Best-of-N Jailbreaking. *arXiv:2412.03556*.

David Glukhov, Ilia Shumailov, Yarin Gal, Nicolas Papernot, and Vardan Papayan. 2024. Position: Fundamental Limitations of LLM Censorship Necessitate New Approaches. In *Proc. ICML*.